



Dissertation on
**“E.C.H.O - Efficient
Contextualized History
Orchestrator”**

*Submitted in partial fulfillment of the requirements for the award of the degree
of*

**Bachelor of Technology
in
Computer Science & Engineering**

UE23CS320B – Capstone Project Phase - 2

Submitted by:

Name: Arjun Sai N	SRN1 PES2UG23CS091
Name: Mohammed Mir Fazlai Ali	SRN2 PES2UG23CS346
Name: Mohammed Shehzaad Khan	SRN3 PES2UG23CS349
Name: Mrigank Jain	SRN4 PES2UG23CS352

Under the guidance of

Dr.Mannuru Shalima Sulthana
Designation
PES University

January - May 2026

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India



PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

‘Title of the project’

is a Bonafide work carried out by

Name: Arjun Sai N
Name: Mohammed Mir Fazlai
Ali

SRN1 PES2UG23CS091
SRN2 PES2UG23CS346

Name: Mohammed Shehzaad
Khan

SRN3 PES2UG23CS349

Name: Mrigank Jain

SRN4 PES2UG23CS352

In partial fulfilment for the completion of sixth-semester Capstone Project Phase - 2 (UE23CS320B) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan. 2026 – May. 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 6th-semester academic requirements in respect of project work.

Dr. M. Shalima

Signature
Dr. Mannuru Shalima Sulthana
Associate Professor

Signature
Dr. Sandesh B J
Chairperson

External Viva

Name of the Examiners

Signature with Date

1. _____

2. _____

DECLARATION

We hereby declare that the Capstone Project Phase - 2 entitled "**E.C.H.O - Efficient Contextualized History Orchestrator**" has been carried out by us under the guidance of **Dr.M.Shalima Sulthana, Associate Professor** and submitted in partial fulfillment of the course requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester January – May 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES2UG23CS091 ARJUN SAI N



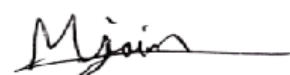
PES2UG23CS346 MOHAMMED MIR FAZLAI ALI



PES2UG23CS349 MOHAMMED SHEHZAAD KHAN



PES2UG23CS352 MRIGANK JAIN



ACKNOWLEDGEMENT

I would like to express my gratitude to **Dr. M.Shalima Sulthana** , Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS320B - Capstone Project Phase-2

I am grateful to all Capstone Project Coordinators, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Sandesh B J, Professor & Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University, and Prof. Nagarjuna Sadineni, Pro-Vice Chancellor, PES University, for providing me with various opportunities and enlightenment every step of the way. Finally, Phase II of the project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

In day to day digital work, most people move between several applications such as Chrome, Gmail and YouTube. We often view something important while browsing or while watching a video, but after a few days, it becomes difficult to remember where exactly we saw it. Usually, instead of searching back through mails or history tabs, we simply start the search again. Because of this common behaviour, we thought of building a system that can remember these digital traces in a meaningful manner and help the user retrieve them when needed.

Echo is basically designed to act as a simple digital memory, not just a history collector. It looks at what the user was probably trying to do at that time. And then it relates all those activities together. So let's say someone searched online, and then watched a tutorial, and then got an email about the same topic. Echo tries to link all three together.

The system is mainly built to run inside the Chrome browser through an extension. It can also use cloud support if required. Storage preference is left to the user so they can keep the data local by default. Basic analytics and simple visuals are added so the user can see how their time is being spent online. And as digital content keeps increasing each year, recalling previous interactions has become a challenge. Something like Echo can basically reduce that effort, so users can go back to what they engaged with earlier. And in the future, the system can get better to support more sources and give deeper insights based on user behaviour.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9
2.	PROBLEM DEFINITION	12
3.	DATA	15
3.1	Overview	15
3.2	Dataset	16
3.3	Data Preprocessing	17
4.	DESIGN DETAILS	19
4.1	Novelty	19
4.2	Innovativeness	20
4.3	Interoperability	21
4.4	Performance	22
4.5	Security	23
4.6	Reliability	24
4.7	Maintainability	25
4.8	Portability	26
4.9	Legacy to Modernization	26
4.10	Reusability	27
4.11	Application Compatibility	27
4.12	Resource Utilisation	28
5.	HIGH LEVEL SYSTEM DESIGN / SYSTEM ARCHITECTURE	29

6.	DESIGN DESCRIPTION	33
6.1	Master Class Diagram	33
6.2	ER Diagram	34
6.3	User Interface Design	35
6.4	Report Layouts	36
6.5	External Interfaces	36
6.6	Packaging and Deployment Diagram	37
7.	TECHNOLOGIES USED	38
8.	IMPLEMENTATION AND PSEUDOCODE	41
9.	CONCLUSION OF CAPSTONE PROJECT PHASE 2	49
10.	PLAN OF WORK FOR CAPSTONE PROJECT PHASE 3	52
	REFERENCES/BIBLIOGRAPHY	55
	APPENDIX A — DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	57
	APPENDIX B — USER MANUAL (OPTIONAL)	—

LIST OF TABLES

Table No.	Title	Page No.
7.1	Complete Technology Stack	38

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	System Architecture — Layered flow from Data Sources through all layers	29
5.2	Module Interaction Diagram	31
6.1	Master Class Diagram	33
6.2	Entity-Relationship Diagram for all 16 tables	34
6.3	Chrome Extension Popup Mockup	35
6.4	Deployment Diagram	37

Chapter 1

INTRODUCTION

1.1 Background

The modern digital user generates a huge volume of activity across multiple platforms every single day. That means receiving and sending emails, browsing research articles, watching educational and entertainment videos, and then switching between all of these continuously. Each platform gives its own native search and history capabilities but none of them communicate with each other. So let's say a user watched a YouTube tutorial, and then read three GeeksForGeeks articles, and then got a confirmation email from a company. All of that is in the context of preparing for a job interview. That user has no way to get all of that as one complete, searchable body of information a week later. The data is fragmented across Gmail, Chrome history, and YouTube watch history and there's no cross-source retrieval mechanism for it.

Echo is a local-first, intent-aware personal memory system built to solve exactly this problem.

So it captures a user's digital activity across Gmail, Chrome, and YouTube, and then enriches it through a background processing pipeline, stores it in a unified local database, and enables natural language conversational search across all three sources at the same time. The system also goes through digital time usage data with a non-judgmental wellbeing module and supports multi-turn conversational queries that resolve contextual references from prior turns.

1.2 Scope of Phase 2

So Phase 2 of Echo covers the complete implementation of the system's intelligence and data layer. That is basically from the initial high-level design all the way through the full delivery of all the core backend modules.

The phase first started with the completion of the high-level design and the architecture, and that includes the 16-table PostgreSQL schema deployed on Neon.tech, OAuth 2.0 authentication configuration for Gmail and YouTube APIs, and then the GitHub repository setup with shared access across the team.

So data ingestion was implemented across all three sources. The Gmail Connector captures all emails via Gmail API OAuth 2.0 polling, stores them in PostgreSQL with thread-level context, and then tracks engagement time for opened emails using the Chrome Extension. The Chrome Connector implements a custom Manifest V3 Chrome Extension with a two-phase intent gate that basically distinguishes genuine engagement from accidental page visits using foreground time, scroll depth, interaction count, and Redis-backed revisit detection. The YouTube Connector detects videos by URL pattern in the extension, tracks foreground playback time, fetches metadata via YouTube Data API v3, and classifies Shorts separately.

The Storage Engine was refactored from direct per-connector PostgreSQL writes into a centralised module with a single atomic transaction interface. And the Enrichment Pipeline was implemented as a background worker that does text cleaning, four-stage system group classification, embedding generation using all-MiniLM-L6-v2, and FAISS indexing.

The Retrieval and Synthesis Engine was implemented as a nine-node LangGraph stateful graph with exactly two LLM calls per query — one for intent parsing and one for answer synthesis — and it also has a self-correcting evaluation loop and multi-turn conversation memory via LangChain's PostgreSQLChatMessageHistory. And the known schema discrepancies from the early implementation were resolved, so that includes the removal of a redundant `gmail_memory` table and correction of YouTube category ID storage.

1.3 Report Organisation

This is how the report is organised. Chapter 2 defines the problem Echo addresses. Chapter 3 describes the data sources, dataset characteristics, and the preprocessing approach. Chapter 4 covers the design quality attributes. Chapter 5 presents the high-level system architecture. Chapter 6 gives the complete detailed design descriptions including class structure, ER relationships, and interface specifications. Chapter 7 lists the complete technology stack. And Chapter 8 presents the implementation with pseudocode for the key components.

Chapter 9 concludes Phase 2 and Chapter 10 gives the plan for Phase 3.

Chapter 2

PROBLEM DEFINITION

2.1 The Core Problem

Basically the digital information a user consumes is trapped in silos right now. Gmail stores emails. Chrome stores browsing history. YouTube stores watch history. And each of these is independent, non-interoperable, and limited to keyword-based search within its own data. So suppose a user wants to find “everything I studied about operating systems after my TechCorp interview email” — they have to manually search Gmail for the email, separately search Chrome history for the browsing activity, and separately try to recall YouTube videos. There’s no way to do this as one complete cross-source query in one place.

And this fragmentation problem gets significantly worse over time. As digital activity accumulates, the signal-to-noise ratio goes down because the user can’t really tell the difference between content they genuinely engaged with and content they accidentally opened for two seconds.

So basically existing tools either capture everything without any filter and create noise, or they make the user manually curate things and that adds friction. And neither of those actually serves what the user really needs from a memory system.

2.2 Secondary Problems

Beyond the retrieval problem, two secondary problems motivated Echo’s design.

The first is the absence of engagement-aware filtering. So the Chrome History API records every URL visited and it doesn’t care whether the user actually read the page or just accidentally clicked a link and closed it right away. And this results in thousands of records per month that are basically semantically meaningless — a redirect page, an accidentally clicked link, a tab opened and forgotten. So a memory system built on raw browsing history is not really a memory system, it’s basically just an indiscriminate surveillance log. What we actually need is a filter that can tell the difference between a user opening an article for two seconds and a user reading it for ten minutes.

These two interactions basically carry different intent signals and they should not be treated the same way. Echo handles this through per-source intent gates that use engagement signals — foreground time, scroll depth, interaction count, and revisit history — to only let in content the user actually paid attention to.

The second is the absence of non-judgmental time reflection. So existing screen time and productivity tools give you scores, warnings, comparisons to population norms, and color-coded assessments of how you used your time. And research on behaviour change consistently shows that prescriptive and judgmental interfaces increase user anxiety and a lot of times produce avoidance rather than reflection. So a user who spent ninety minutes on YouTube Shorts doesn't really get anything useful from being told their "wellbeing score dropped." What they actually need is just a factual description — how much time was spent, on what type of content, at what time of day, and whether they themselves have flagged similar sessions as regrettable before.

So Echo's wellbeing module just shows descriptive patterns and it doesn't do any scoring, blocking, or judging. And the regret system is completely user-declared, meaning Echo never decides what is regrettable. Only the user can.

2.3 Problem Statement

So the goal is to design and implement a local-first personal memory system that does a few things. First, it captures digital activity from Gmail, Chrome, and YouTube with intent-aware filtering that basically distinguishes genuine engagement from accidental exposure. Next, it enriches the captured content through background semantic processing. Then it enables natural language cross-source search using a deterministic retrieval pipeline with LLM-based intent parsing and answer synthesis. And it also supports multi-turn conversational queries with persistent session memory. Finally, it reflects on time usage patterns in a descriptive and non-judgmental way. So all personal data has to stay on the user's local machine. And the system has to be architecturally transparent and academically defensible.

Chapter 3

DATA

3.1 Overview

Echo does not use a static dataset.

The data is basically the user's own live digital activity, and it's captured in real time from three sources. First, Gmail via the Gmail API OAuth 2.0, then Chrome browsing via a custom Manifest V3 Chrome Extension, and then YouTube via the YouTube Data API v3 combined with extension-based playback detection. And all the captured data is stored in a PostgreSQL database hosted on Neon.tech during development and then migrated to local PostgreSQL at submission time via a single `pg_dump` command.

The data model is unified, meaning every email, web page, and YouTube video is represented as a `memory_items` record. And it's extended by source-specific subtype tables like `gmail_metadata`, `chrome_metadata`, and `youtube_metadata`, and also a shared engagement table called `memory_engagement`. So this unified abstraction is basically what makes cross-source search work.

3.2 Dataset

So since Echo captures the user's own live activity, there is no pre-existing training dataset. But for development and testing, the team used controlled test data including synthetic emails, manually browsed web pages on known topics, and YouTube videos watched under test conditions. So the test scenarios are documented in the architecture specification, and they include scenarios for the intent gate, meaning pages opened briefly versus pages read thoroughly. And then revisit detection, which is the same URL visited on separate occasions within 24 hours. And also email thread enrichment for reply emails with no subject context, and attachment handling for PDFs embedded in emails.

For demonstrating the system, a representative set of around 200 memory items across the three sources was created. And it covers the categories work, study, entertainment, personal, and misc. And this is basically sufficient to demonstrate the classification pipeline, FAISS semantic search, and LangGraph retrieval.

So the estimated data volume for a typical student user is around 1.25 MB per day. And that's based on 20 emails per day at roughly 20 KB each, then 35 web pages per day at roughly 20 KB each, and then 10 YouTube videos per day at roughly 15 KB each.

For a full academic year, this comes to around 450 MB, which is well within the storage capacity of any modern laptop.

3.3 Data Preprocessing

So the raw data ingested from the three APIs is not immediately suitable for semantic search. And each source needs source-specific preprocessing before embedding generation. All preprocessing is done by the Enrichment Pipeline, basically ENP, as a background worker and not on the hot ingestion path.

For Gmail items, preprocessing involves stripping HTML tags, removing tracking pixels embedded in email bodies, and then removing email signatures using pattern matching on common farewell phrases and line structures. And also removing quoted reply content identified by lines beginning with the greater-than character. The result is a `clean_body` field that has only the readable primary content of the email. And then the embeddable text is constructed by concatenating the email subject, the clean body, thread keywords extracted from prior emails in the same thread, and a sender domain hint.

For Chrome items, preprocessing uses the Readability algorithm, the same algorithm used by Firefox Reader View, to extract the main article content from the stored HTML. And this strips navigation elements, sidebars, advertisements, footer links, and comment sections, leaving only the primary article text with headings preserved. So the embeddable text is constructed from the page title, extracted headings, a content snippet, and the domain name.

For YouTube items, preprocessing tries to fetch the auto-generated transcript from YouTube's caption endpoint on a best-effort basis, since many videos don't have transcripts available. And the embeddable text is constructed from the video title, description, channel name, a category hint derived from the YouTube Data API category ID, and the first 500 characters of the transcript if available.

After preprocessing, each item's embeddable text is encoded using the all-MiniLM-L6-v2 model from the sentence-transformers library [3], and this basically produces a 384-dimensional float32 vector.

This vector is added to the FAISS index [4] and a corresponding row is inserted into the embed-

ding_index table, and that row records the model version, vector dimension, and active status. And the preprocessed flag on memory_items is set to TRUE only after all of these steps complete successfully.

Chapter 4

DESIGN DETAILS

4.1 Novelty

Echo introduces a few design decisions that make it different from existing personal information management tools. The most significant one is the combination of intent-aware ingestion with deterministic retrieval. So basically most personal data capture tools either capture everything, which creates noise, or they require manual curation, which creates friction. Echo uses per-source intent gates, and that is a two-phase gate for Chrome items requiring a minimum of five seconds of exposure and at least one of several engagement signals, and a twenty-second sustained playback threshold for YouTube items, and this is what automatically distinguishes genuine engagement from accidental exposure without needing any user action.

The second novel element is the LangGraph-based deterministic retrieval architecture. Instead of using a free-roaming agentic LLM that selects tools dynamically, Echo uses a fixed nine-node stateful graph. The LLM is called exactly two times per query, first for intent parsing and then for answer synthesis, and all the retrieval logic in between is deterministic Python. And this is what makes every retrieval decision visible, debuggable, and academically defensible. The self-correcting evaluation loop widens the search scope up to three times before returning a no-results message, and that is a practical engineering solution to the problem of sparse databases in early system usage.

The third novel element is the four-stage cascading classifier for system group assignment. It uses structural API signals, domain lookup, nearest centroid comparison against FAISS embeddings already computed for search, and LLM batch fallback, in that order. And this architecture is what gets to approximately 90% classification without an LLM call while still maintaining accuracy through passive centroid improvement over time.

4.2 Innovativeness

The architecture of Echo represents an innovative integration of several technologies. These are individually well-established but not previously combined in this configuration for personal memory

management. And the combination of a Chrome Extension for engagement signal capture, a FAISS vector store for semantic search, a LangGraph stateful graph for retrieval orchestration, LangChain's PostgresChatMessageHistory for conversation persistence, and a plug-and-play LLM configuration through LangChain's ChatModel abstraction is what makes this system simultaneously privacy-preserving, semantically capable, and provider-agnostic.

The passive centroid improvement mechanism in Stage 3 of the classifier is an innovative use of unsupervised learning principles. So by recomputing the centroid vector for each system group monthly from the confirmed items, meaning the ones classified as structural or user-overridden, the classifier's accuracy for ambiguous items gets better over time. And this happens without any explicit retraining, without storing labelled training data, and without any cost beyond the monthly average computation.

4.3 Interoperability

So Echo is basically designed with interoperability as a first-class concern. The three data sources, Gmail, Chrome, and YouTube, are all integrated through their official published interfaces. Gmail is accessed via Gmail API v1 using OAuth 2.0 with standard refresh token handling. YouTube metadata is fetched via YouTube Data API v3. And Chrome activity is captured via a Manifest V3 Chrome Extension using standard W3C browser APIs for tab events, window focus, and content script injection.

The LLM integration is fully provider-agnostic through LangChain's ChatModel abstraction layer. So the LLM configuration is basically a single dictionary in config.py and it specifies the provider, the API key, the parser model, and the synthesizer model. LangChain wrappers handle the provider-specific API call differences transparently. And during development, the system is configured with Gemini 2.5 Flash, stored in the project .env file. And switching providers is basically just changing one config entry, and it doesn't touch any of the application logic at all.

The Storage Engine gives a stable interface to all connectors through the store_memory_item() function, which takes the source type, source ID, metadata, and engagement data and handles all the database write logic internally.

4.4 Performance

So the performance targets for Echo are defined per operation type in the architecture specification. Email ingestion to PostgreSQL has to complete within 500 milliseconds. The Chrome intent gate decision has to complete within 100 milliseconds. And enrichment pipeline processing per item has

to complete within 5 seconds, which is fine because it runs in the background and doesn't block ingestion. The PostgreSQL search query has to complete within 200 milliseconds for a database of 10,000 items. FAISS semantic search for 20 candidates from 50,000 items has to complete within 50 milliseconds. And LLM intent parsing using Gemini 2.5 Flash is expected to take 1 to 3 seconds, and LLM synthesis 2 to 5 seconds. And the full end-to-end query response time target, excluding attachment fetching, is under 8 seconds.

These targets are achieved through a few specific design decisions. The PostgreSQL pre-filter step in the RSE reduces the FAISS search space from the entire index to a candidate set that matches structural filters, so FAISS never has to go through the entire index. The partial index on `memory_items` where `preprocessed` is `FALSE` makes sure the enrichment pipeline's polling query stays fast even as the table grows. And the FAISS flat L2 index is basically fast enough at the expected scale of 50,000 to 100,000 items for this project.

4.5 Security

So Echo's security model is built around local-first data storage. All the personal data, meaning email content, browsing history, YouTube activity, conversation history, is stored only in PostgreSQL on the user's local machine.

The only external transmissions are the OAuth API calls to Gmail and YouTube using standard refresh token flows, and also the query text plus retrieved result snippets that get sent to the LLM API for intent parsing and synthesis. So the LLM API never receives the full email bodies, full page content, video transcripts, the user's complete database, or FAISS vectors.

OAuth tokens for Gmail and YouTube are stored encrypted, and separately from the main database. The Chrome Extension does not run in incognito mode by design, and this is enforced at the Manifest V3 level. So the extension captures no typed text, no form inputs, no passwords, and no application page content from tools like Slack, Jira, or Notion. Application pages are tracked only for session duration, and their content is never extracted or stored.

During development, the Neon PostgreSQL instance has only synthetic test data in it, so it doesn't have the developers' real personal emails, browsing history, or watch history.

4.6 Reliability

So basically, reliability in Echo is designed at the data ingestion layer. The `UNIQUE(source_type, source_id)` constraint on `memory_items` with `ON CONFLICT DO NOTHING` makes it so that ingestion can be restarted at any time without creating duplicate records. And this is the primary crash-safety guarantee, meaning if a connector crashes mid-run and restarts, it can safely re-process the same records without data corruption [1].

Every ingestion write is wrapped in a single atomic transaction, so that's `BEGIN`, `INSERT` into `memory_items`, `INSERT` into the source-specific metadata table, `INSERT` into `memory_engagement`, `COMMIT`, and this follows Pattern A defined in the database design document. So partial ingestion is structurally prevented, and a transaction that fails midway is rolled back entirely.

And then the enrichment pipeline is designed to tolerate individual item failures. If cleaning or embedding fails for one item, the error is logged and the pipeline just continues to the next item. And the preprocessed flag stays `FALSE` for that failed item, so it will be retried on the next polling cycle.

The LangGraph retrieval graph is reliable by construction. The self-correcting evaluation loop retries retrieval with progressively wider parameters up to three times. And after three failures, the `no_results_found` node returns a structured message explaining what was searched, rather than going into an error state or silent failure.

4.7 Maintainability

So basically the codebase is organised into eight modules with clearly defined responsibilities and interfaces. Each module is independently testable. The Storage Engine presents a stable interface so that schema changes, if any, are absorbed internally without requiring changes to the three connectors. The LangGraph graph structure separates routing logic in `graph_routing.py` from node implementations in `graph_nodes.py` and the graph assembly in `retrieval_engine.py`, so you can modify a single node without touching the graph topology.

The four-stage classifier is basically implemented as a single orchestrating function which is `classify_system_group`, and it calls four independent stage functions. And adding a new rule to Stage 1 or a new domain entry to Stage 2 requires no structural changes to the classifier pipeline.

And the LLM configuration is entirely externalised to `config.py` and the project `.env` file. So no model names, API endpoints, or provider-specific logic appear anywhere except in the LangChain provider

factory in `llm_synthesizer.py`.

4.8 Portability

So basically Echo is designed to run on any machine with Python 3.10 or later, a Chrome browser, and internet connectivity for the initial OAuth setup. And the production deployment is entirely local, meaning no cloud services are required after submission.

The `pg_dump` migration from Neon.tech to local PostgreSQL is basically a single command that transfers the complete schema, all the data, all indexes, and all constraints.

FAISS is a local in-memory index loaded from a file on disk. It runs without a server process, without network connectivity, and without any cloud dependency. And the sentence-transformers model which is all-MiniLM-L6-v2 is downloaded once and cached locally, so subsequent runs don't require internet access for embedding generation.

4.9 Legacy to Modernization

So the Chrome History API was considered as a data source during early design but was rejected because it gives only historical URL timestamps with no engagement signals. And Echo's custom Manifest V3 Chrome Extension is basically a modernisation of that approach. So it gives real-time engagement signals including foreground time, scroll depth, interaction count, and revisit detection, and none of those are available through the History API.

This architectural decision is documented in the scope change log as a deliberate upgrade from the original approach.

The earlier design for the retrieval engine was a linear seven-step fallback chain. And that was replaced with a LangGraph stateful graph. So LangGraph gives explicit state management, conditional routing, and a loop structure that the linear chain could not support, and that is basically important for multi-turn conversation resolution where the query in Turn 2 references entities from Turn 1.

4.10 Reusability

So the Storage Engine module is designed for reusability. The `store_memory_item()` function accepts a `source_type` parameter and dispatches to the appropriate metadata table, meaning it handles Gmail, Chrome, and YouTube items through a single interface. So if a fourth data source needs to be added in

a future version, that would require only a new metadata table and a new branch in the dispatch logic, with no changes to the function signature.

The four-stage classifier is fully reusable. It basically operates on a memory item and its embedding vector, with no dependency on which connector produced the item. And the centroid improvement mechanism is self-maintaining, meaning it doesn't require retraining when new item types are added.

And the plug-and-play LLM configuration through LangChain's ChatModel abstraction means that any component using LLM calls can switch providers with no changes to the application code.

4.11 Application Compatibility

Echo is compatible with any Chromium-based browser that supports Manifest V3 extensions. The backend requires Python 3.10 or later and PostgreSQL 15 or later, and it's currently running on PostgreSQL 16 on Neon.tech. And the React dashboard is compatible with all modern browsers.

The sentence-transformers library and FAISS are compatible with Linux, macOS, and Windows, and for FAISS on Windows that's via WSL2.

4.12 Resource Utilisation

So PostgreSQL storage usage is estimated at around 450 MB per year for a typical student user [1].

So the FAISS index size is around 75 MB for 50,000 items at 384 dimensions. And Redis usage is bounded at around 10 MB for active revisit signals and up to 200 KB per cached attachment extract, with both getting evicted automatically by TTL.

CPU usage during enrichment is basically bounded by the batch size of the polling loop. The enrichment pipeline processes items with a 10-second polling interval so it doesn't compete with foreground tasks for CPU resources. And embedding generation using the all-MiniLM-L6-v2 model on CPU takes around 20 to 50 milliseconds per item. LLM API calls are the highest-latency operations in the system but they're network-bound, not CPU-bound.

Chapter 5

HIGH LEVEL SYSTEM DESIGN / SYSTEM ARCHITECTURE

5.1 Architecture Overview

So Echo follows a layered modular monolithic architecture with a three-layer storage system feeding into a LangGraph-based retrieval and synthesis pipeline, served through a React web dashboard and a Chrome Extension popup.

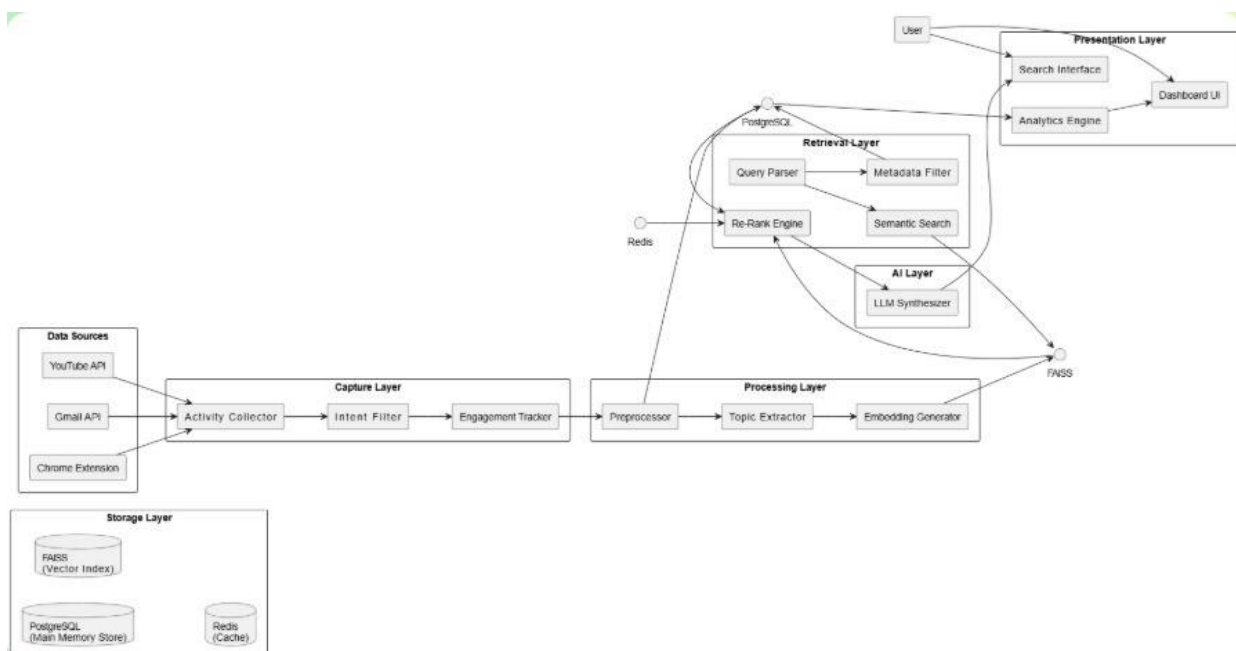


Figure 5.1: System Architecture — Layered flow from Data Sources (Gmail API, Chrome Extension, YouTube Data API) through the Capture Layer, Processing Layer, Storage Layer, Retrieval Layer, and AI Layer to the Presentation Layer

The three data sources feed into source-specific connectors. The Gmail Connector (GMC) uses Gmail API OAuth 2.0 polling. The Chrome Connector (CHC) uses a custom Manifest V3 Chrome Extension with a two-phase intent gate. And the YouTube Connector (YTC) uses URL-based video detection in the extension and then fetches metadata using the YouTube Data API v3. All three connectors route their writes through the Storage Engine (STE), which basically executes atomic transactions against the PostgreSQL schema.

So basically, raw ingested items have the preprocessed flag set to FALSE. And the Enrichment Pipeline (ENP) runs as a background worker, polling for unprocessed items and then applying text cleaning, system group classification, embedding generation, and FAISS indexing. Once ENP completes all the steps for an item, the preprocessed flag is set to TRUE, and that's what makes the item available for semantic search.

User queries get routed through the FastAPI gateway to the LangGraph RSE. The RSE executes a nine-node stateful graph. First, `parse_intent` calls the LLM to produce structured JSON. Next, `postgres_search` executes a dynamic SQL query. Then `faiss_search` performs vector similarity search on the candidate set. `evaluate_quality` deterministically assesses result quality. `widen_scope` progressively relaxes filters on weak results up to three times. And then `check_attachments` and `fetch_attachment` handle on-demand attachment extraction. Then `synthesize` calls the LLM to produce the final answer. And finally `no_results_found` handles the exhausted-retry case.

The LLM is called exactly two times per query.

5.2 Module Architecture

Echo is composed of eight modules: GMC (Gmail Connector), CHC (Chrome Connector), YTC (YouTube Connector), STE (Storage Engine), ENP (Enrichment Pipeline), RSE (Retrieval and Synthesis Engine), WBA (Digital Wellbeing and Analytics), and UIL (User Interface Layer). Each module has a defined set of responsibilities, key files, and inter-module interface contracts.

5.3 The LangGraph RSE Graph

And the graph topology is fixed and deterministic.

The only conditional branching points are: after `evaluate_quality`, it routes to `check_attachments` on strong results or to `widen_scope` on weak or empty results. And then after `widen_scope`, it routes back to `postgres_search` if `attempt_count` is less than 3 or to `no_results_found` if attempts are exhausted. And after `check_attachments`, it routes to `fetch_attachment`, `fetch_api`, or directly to `synthesize` depending on the flags set by `parse_intent`.

5.4 Three-Layer Storage

PostgreSQL on Neon.tech (migrating to local at submission) is the only permanent storage layer. It holds all 16 tables and is basically the only layer that can't be rebuilt from another source. FAISS

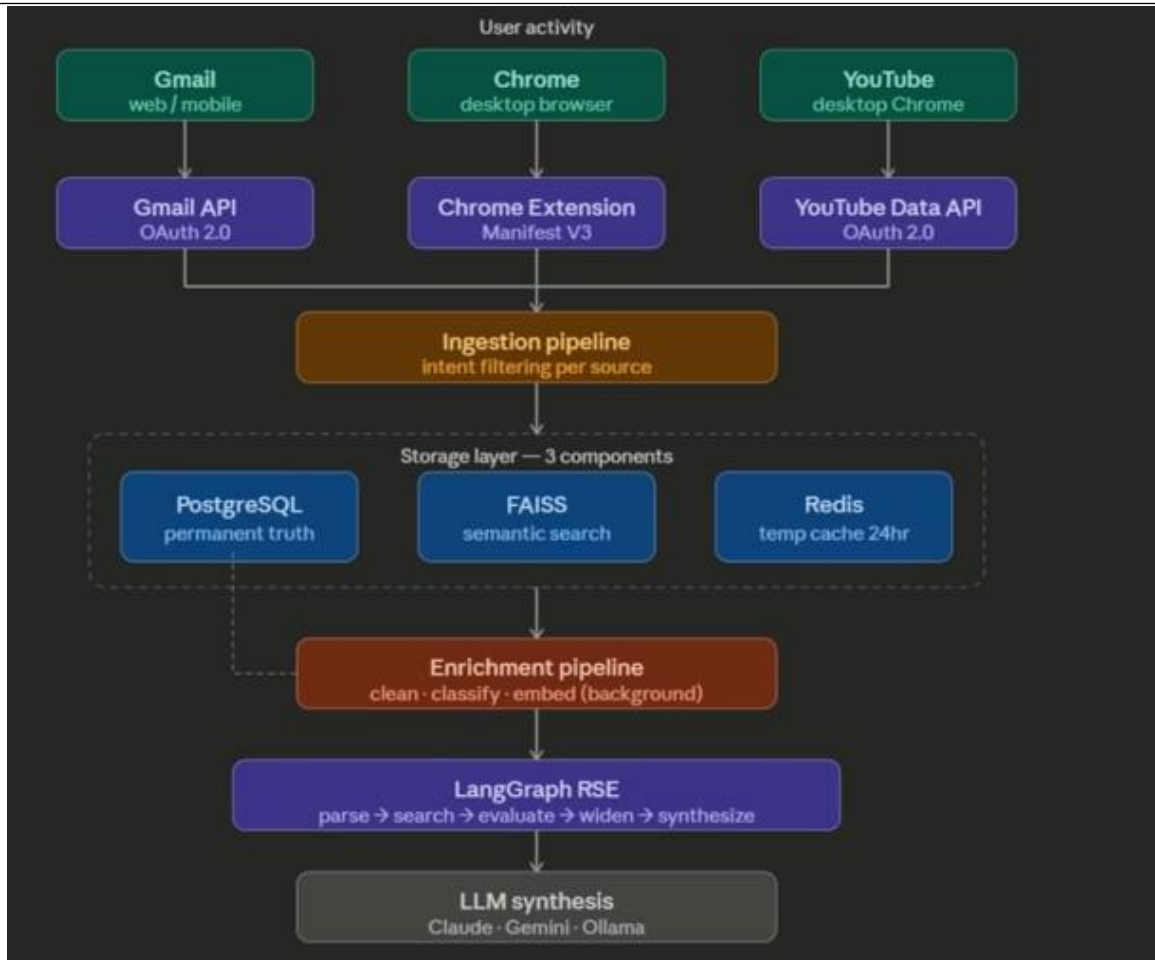


Figure 5.2: Module Interaction Diagram — showing GMC, CHC, and YTC writing to STE; STE writing to PostgreSQL, FAISS, and Redis; ENP polling from STE; RSE querying STE and producing answers for UIL; WBA querying STE for analytics

is a local vector index that can be fully rebuilt from the `embedding_index` table in PostgreSQL by re-encoding the `embeddable_text` stored there. And Redis holds only time-bounded data, so that's revisit signals with a 24-hour TTL and attachment text extracts with a 1-hour TTL, and it's entirely disposable.

So the Redis connection URL in `redis_manager.py` is the only config change you need to switch between Upstash and a local Redis instance.

Chapter 6

DESIGN DESCRIPTION

6.1 Master Class Diagram

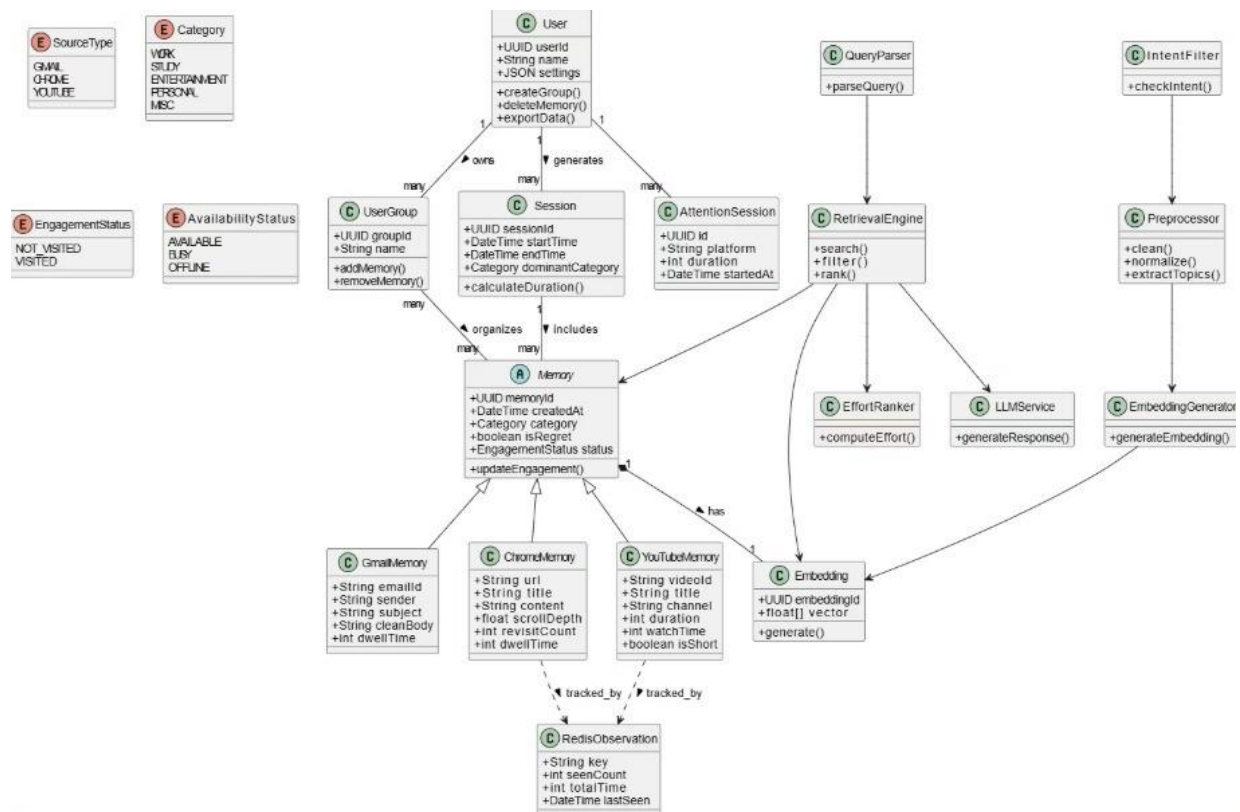


Figure 6.1: Master Class Diagram — showing the eight module classes, key methods, inter-module dependencies, and EchoState TypedDict as the data contract between LangGraph nodes

So the key class relationships are basically: StorageEngine is depended on by GmailConnector, ChromeConnector, and YouTubeConnector via the `store_memory_item()` interface. And EnrichmentPipeline depends on StorageEngine for polling unprocessed items and writing enriched results, and it also depends on EmbeddingGenerator and SystemGroupClassifier internally. And the RetrievalEngine, so that's the LangGraph graph, depends on StorageEngine for `postgres_search` and `faiss_search`, and on LLMProvider via the LangChain ChatModel factory for `parse_intent` and `synthesize`.

And the APIGateway depends on RetrievalEngine and WellbeingAnalytics to serve the frontend requests.

6.2 ER Diagram

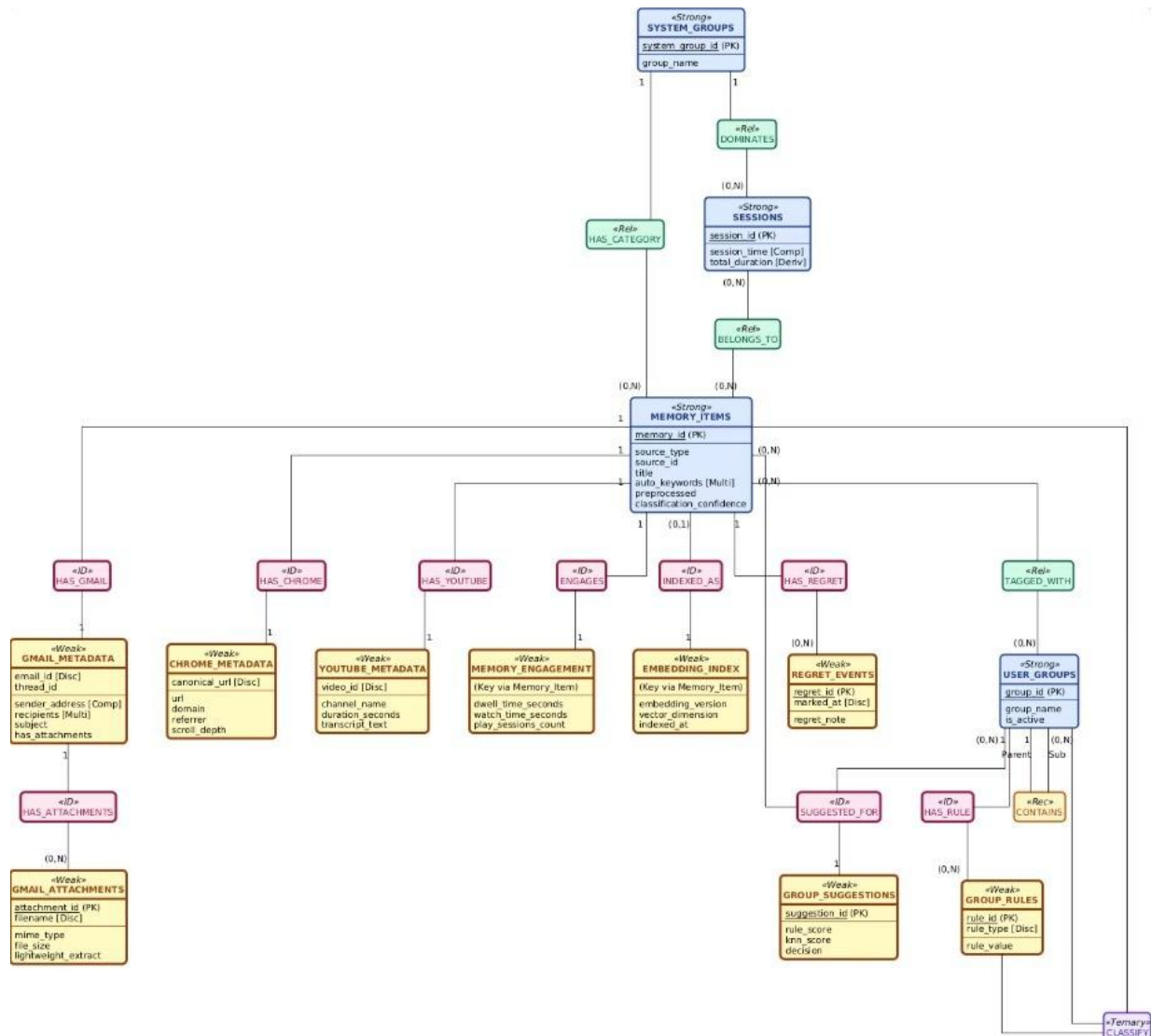


Figure 6.2: Entity-Relationship Diagram for all 16 tables — showing strong entities, weak subtypes, associative bridges, the UNIQUE(source_type, source_id) constraint on memory_items, and the NOT NULL constraint on system_group_id

So the key relationships are basically: memory_items has 1:1 relationships with gmail_metadata, chrome_metadata, youtube_metadata, memory_engagement, and embedding_index, and that last one is partial, so only after preprocessing. And then memory_items has a 1:N relationship with regret_events, meaning each toggle is a new row. And memory_items also has M:N relationships with user_groups via memory_user_groups and with sessions via session_memory_map. And user_groups has 1:N relationships with group_rules and group_suggestions.

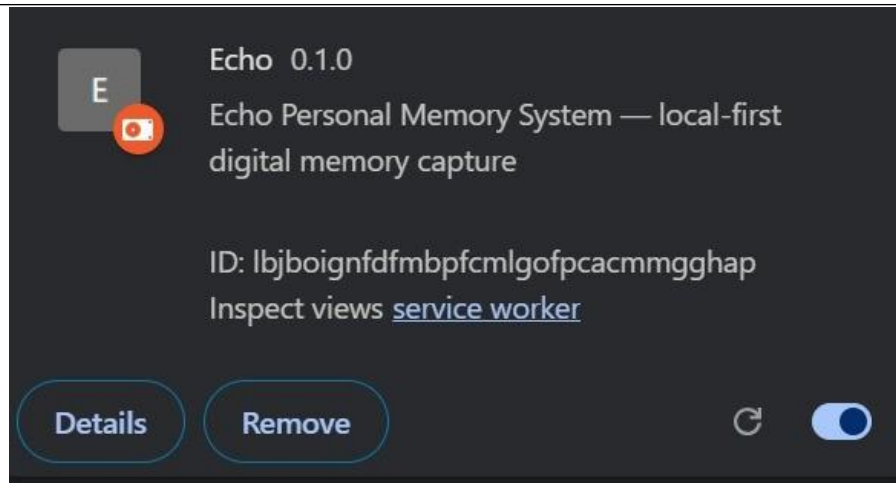


Figure 6.3: Chrome Extension Popup Mockup — compact search bar, last 5 items across all sources with source icon and title, pause/resume tracking toggle, and link to open full dashboard

6.3 User Interface Design

6.4 Report Layouts

The search results from the RSE basically start with the synthesised answer paragraph at the top. And then below it is a numbered list of source citations. So for each citation it has the source type, title, timestamp, and a short excerpt. And for Gmail results, the original email is accessible via a link to Gmail web.

So for Chrome results, the stored URL is clickable. And for YouTube results, the YouTube video URL is linked.

The analytics reports on the dashboard are presented as charts. So the daily summary uses a horizontal stacked bar chart. The time heatmap uses a grid of cells sized by hour of day and day of week, with a colour gradient from light to dark to show activity intensity, and R markers on the cells where regret events occurred.

6.5 External Interfaces

So Gmail API v1 is authenticated via OAuth 2.0, and it is used for email polling and attachment fetching. It requires the `users.messages.list` and `users.messages.get` scopes. And the refresh token is stored encrypted locally.

YouTube Data API v3 is authenticated via OAuth 2.0, and it is used for video metadata fetching, so basically the title, description, channel, duration, and category ID. It requires the `youtube.readonly`

scope. And it is one API call per new video at ingestion time.

The LLM API is accessed through LangChain's ChatModel abstraction layer. It's currently configured with Gemini 2.5 Flash via the `GOOGLE_API_KEY` stored in the project `.env` file. And it's used for intent parsing, so that's LLM Call 1, and answer synthesis, so that's LLM Call 2. Basically only the user's query text, conversation history, and short result snippets get transmitted. No personal data content is sent.

And for the Chrome Extension to Backend, the extension communicates with the FastAPI backend over localhost on a configurable port. So the default is 127.0.0.1 rather than 0.0.0.0, and that is so that it works on university and office networks that block external port binding.

6.6 Packaging and Deployment Diagram

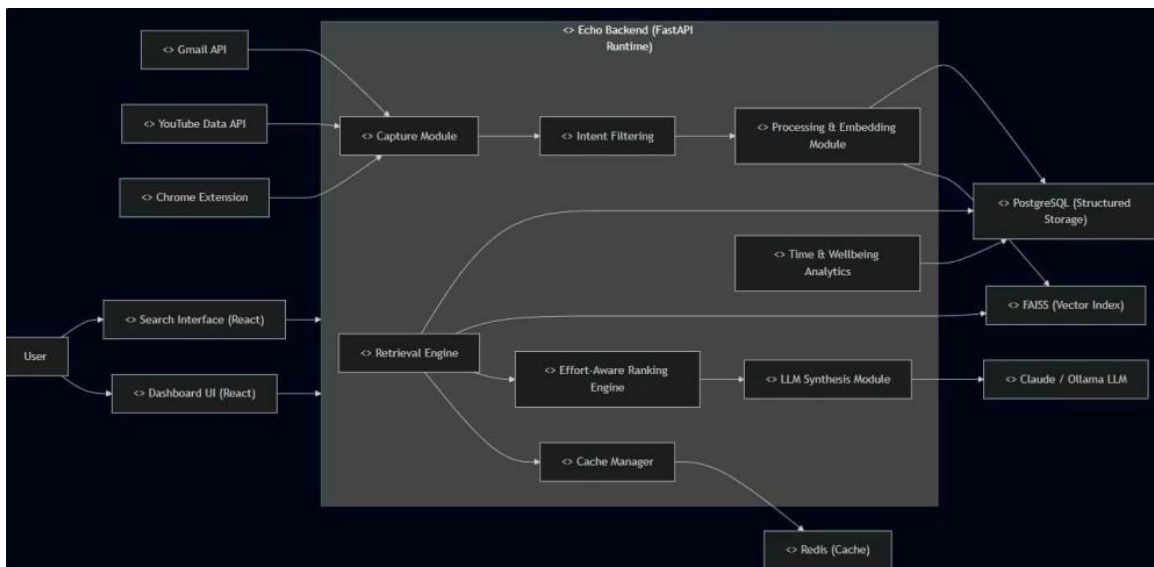


Figure 6.4: Deployment Diagram — showing the user's laptop running the Python backend, local PostgreSQL (Neon.tech during development), local FAISS index, local Redis (Upstash during development), the Chrome browser with the MV3 Extension, and the React dashboard served by FastAPI

During development, PostgreSQL and Redis are hosted on cloud services, meaning Neon.tech and Upstash, and that is basically to enable parallel development across the four-person team. And at submission, a single `pg_dump` command migrates the complete database to local PostgreSQL. So the `pg_dump` export and `psql` import command pair transfers all the schema, data, indexes, constraints, and foreign key relationships exactly, and no schema modifications are needed post-migration.

Chapter 7

TECHNOLOGIES USED

So the complete technology stack for Echo Phase 2 is listed below in Table 7.1.

Table 7.1: Complete Technology Stack

Layer	Technology	Purpose
Storage	PostgreSQL 16+ (Neon.tech)	Primary permanent database — 16 tables, 19 indexes, full constraint set
Storage	FAISS (faiss-cpu)	Local semantic vector index — 384-dim flat L2, rebuilt from PostgreSQL
Storage	Redis 7+ (Upstash)	Ephemeral signal cache — revisit detection (24hr TTL), attachment cache (1hr TTL)
Backend	Python 3.10+	Primary language for all backend logic
Backend	FastAPI	API gateway serving all frontend and extension routes
Backend	SQLAlchemy 2.x	PostgreSQL ORM with connection pooling
Backend	LangGraph	Stateful directed graph framework for the RSE
Backend	LangChain	ChatModel abstraction, structured output parsing, PostgresChatMessageHistory
Enrichment	sentence-transformers	all-MiniLM-L6-v2 model, 384-dim local embeddings, no API calls
Enrichment	Readability (readability-lxml)	Article content extraction from HTML
Enrichment	PyPDF2 / pdfplumber	PDF text extraction for Gmail attachments
Enrichment	python-docx	DOC and DOCX text extraction
Enrichment	pytesseract + Tesseract	OCR for image-based attachments
Frontend	React 18+ + TailwindCSS 3.x	Web dashboard
Frontend	Recharts	Analytics charts and heatmaps
Extension	JavaScript, Manifest V3	Tab focus, scroll depth, interaction detection, YouTube DOM monitoring
LLM	Gemini 2.5 Flash (LangChain)	Intent parsing and answer synthesis — current dev config
Testing	pytest	Python backend unit tests
Testing	Jest	React and JavaScript unit tests
Testing	Postman	API endpoint testing
Dev Tools	Git / GitHub	Version control with shared team repository
Dev Tools	python-dotenv	Environment variable management from .env

Storage layer: PostgreSQL 16+ via Neon.tech for cloud dev and local PostgreSQL at submission. It is the primary permanent database. So it has all 16 tables, 19 indexes, and the full constraint set.

FAISS which is faiss-cpu latest is the local semantic vector index. It is a 384-dimensional flat L2 index and it gets rebuilt from PostgreSQL via the `embedding_index` table. Redis 7+ via Upstash is the ephemeral signal cache. So it handles revisit detection with 24-hour TTL keys and the attachment text cache with 1-hour TTL. And it uses the standard Redis URL with the `redis-py` client.

Backend: Python 3.10+ is the primary language for all the backend logic. FastAPI latest is the API gateway and it serves all the frontend and extension routes. SQLAlchemy 2.x is the PostgreSQL ORM with connection pooling. LangGraph latest is the stateful directed graph framework for the RSE. It provides the node definitions, conditional edge routing, and state management. LangChain latest is the ChatModel abstraction layer and it handles plug-and-play LLM providers, structured output parsing, and also `PostgresChatMessageHistory` for conversation memory.

Enrichment: `sentence-transformers` latest uses the `all-MiniLM-L6-v2` model to produce 384-dimensional float32 embeddings. It is local inference, meaning no API calls. `Readability` which is `readability-lxml` is for article content extraction from HTML and it is basically equivalent to Firefox Reader View. `PyPDF2` and `pdfplumber` are for PDF text extraction with respect to Gmail attachments. `python-docx` is for DOC and DOCX text extraction. And `pytesseract` with Tesseract is the OCR for image-based attachments.

Frontend: React 18+ with TailwindCSS 3.x is the web dashboard. `Recharts` latest is for the analytics charts and heatmaps.

Chrome Extension: it uses JavaScript with Manifest V3 and it handles tab focus tracking, window focus, scroll depth measurement, interaction detection, and also YouTube DOM monitoring.

LLM provider for the current development configuration: Gemini 2.5 Flash via LangChain's `ChatGoogleGenerativeAI` wrapper and it is configured in the project `.env` as `GOOGLE_API_KEY`. So the architecture is provider-agnostic through LangChain's ChatModel abstraction, meaning switching to any other supported provider requires only a configuration change. The system supports Anthropic Claude, Google Gemini, and locally-hosted Ollama models and you can use them interchangeably.

Development and testing: `pytest` is for Python backend unit tests. `Jest` is for React and JavaScript unit tests. `Postman` is for API endpoint testing. And `Git` is the version control with a GitHub repository shared across the team.

`python-dotenv` is for environment variable management from the `.env` file at the project root.

Chapter 8

IMPLEMENTATION AND PSEUDOCODE

8.1 Storage Engine — Atomic Ingestion Transaction

So the `store_memory_item()` function in `storage_engine.py` basically implements Pattern A from the database design specification. And it is a single atomic transaction that covers the INSERT into `memory_items`, the INSERT into the source-specific metadata table, and also the INSERT into `memory_engagement`.

```
function store_memory_item(source_type, source_id, metadata, engagement):
    BEGIN transaction
    memory_id = generate_uuid_v4()

    INSERT INTO memory_items:
        memory_id, system_group_id, source_type, source_id,
        title, raw_text, auto_keywords,
        preprocessed = FALSE, classified_by = 'pending',
        created_at = metadata.created_at,
        first_ingested_at = NOW()
    ON CONFLICT (source_type, source_id) DO NOTHING

    IF conflict occurred:
        ROLLBACK
        RETURN existing_memory_id

    IF source_type == 'gmail':
        INSERT INTO gmail_metadata:
            memory_id, email_id, thread_id, sender,
            recipients, subject, received_at, has_attachments

    ELSE IF source_type == 'chrome':
```



```
INSERT INTO chrome_metadata:
    memory_id, url, canonical_url, domain,
    scroll_depth = 0.0,
    interaction_count = 0, revisit_count = 0

ELSE IF source_type == 'youtube':
    INSERT INTO youtube_metadata:
        memory_id, video_id, channel_name,
        duration_seconds, is_short, youtube_category_id

INSERT INTO memory_engagement:
    memory_id,
    dwell_time_seconds = 0,
    watch_time_seconds = 0,
    first_opened_at = NULL,
    last_accessed_at = NULL

COMMIT
RETURN memory_id
```

8.2 Enrichment Pipeline — Background Poll Loop

So the `enrichment_pipeline.py` worker polls PostgreSQL every 10 seconds for items where `preprocessed` is equal to `FALSE`. And it processes each one through the complete pipeline.

```
function run_enrichment_loop():
    WHILE True:
        items = SELECT * FROM memory_items
                WHERE preprocessed = FALSE
                LIMIT 50

        FOR each item in items:
            TRY:
                cleaned_text    = text_cleaner.clean(item)
```

```
embeddable_text = construct_embeddable_text(item, cleaned_text)
embedding        = generate_embedding(embeddable_text)
system_group, confidence, stage =
    classify_system_group(item, embedding)

faiss_index.add(embedding, item.memory_id)

INSERT INTO embedding_index:
    memory_id, embedding_version,
    vector_dimension = 384,
    embeddable_text, is_active = TRUE

UPDATE memory_items SET
    system_group_id          = system_group,
    classified_by            = stage,
    classification_confidence = confidence,
    preprocessed             = TRUE
WHERE memory_id = item.memory_id

EXCEPT Exception as e:
    LOG error for item.memory_id
    CONTINUE to next item

SLEEP 10 seconds
```

8.3 Four-Stage System Group Classifier

So the `classify_system_group()` function in `system_group_classifier.py` is what orchestrates the four-stage cascade in detail.

```
function classify_system_group(item, embedding):

    // Stage 1: Structural Signals
    result = structural_classify(item)
```

```
IF result is not None:
    RETURN result, 1.0, 'structural'

// Stage 2: Domain Lookup (Chrome only)
IF item.source_type == 'chrome':
    result = domain_classify(item.domain)
    IF result is not None:
        RETURN result, 0.95, 'domain'

// Stage 3: Nearest Centroid
category, score, clarity = nearest_centroid_classify(embedding)
IF clarity == 'confident':
    RETURN category, score, 'centroid'

// Stage 4: LLM Batch Fallback
ADD item to pending_llm_batch
result = llm_batch_classify(item)
RETURN result, 0.90, 'llm'

// Fallback -- should never reach here
RETURN 'misc', 0.0, 'fallback'
```

8.4 LangGraph RSE — parse_intent Node

So basically the parse_intent node in graph_nodes.py is what sends the user query and conversation history to the LLM via LangChain's ChatModel interface and then parses the structured JSON output.

```
function parse_intent_node(state: EchoState) -> EchoState:
    system_prompt = """
    You are Echo's query parser. Output valid JSON only.
    Always search all sources unless source is explicitly specified.
    Set fetch_attachment=true only if the user mentions:
        paper, PDF, document, file, attachment.
    Set fetch_api=true only if the user mentions:
```

latest, today, just now, received today.

Include time_filter only for explicit time references.

[FEW-SHOT EXAMPLES]

Query: 'find my TechCorp interview email'

```
Output: {"sources":["gmail"],"time_filter":null,
        "fetch_attachment":false,"fetch_api":false,
        "query_clean":"TechCorp interview","is_ambiguous":false}
```

"""

```
messages = [SystemMessage(system_prompt)] +
            state.conversation_history +
            [HumanMessage(state.user_query)]
```

```
response = chat_model.invoke(messages)
parsed_json = json.loads(sanitise_json_response(response.content))
```

```
state.parsed_intent = parsed_json
RETURN state
```

8.5 LangGraph RSE — postgres_search Node

The postgres_search node is what dynamically assembles and executes a SQL query based on the parsed_intent JSON.

```
function postgres_search_node(state: EchoState) -> EchoState:
    intent = state.parsed_intent

    base_query = """
    SELECT m.memory_id, m.source_type, m.title, m.created_at,
           m.auto_keywords, m.system_group_id,
           me.dwell_time_seconds, me.watch_time_seconds,
           me.last_accessed_at,
           cm.url, cm.domain, cm.scroll_depth,
```

```

        gm.sender, gm.subject, gm.has_attachments,
        ym.channel_name, ym.is_short
FROM memory_items m
LEFT JOIN memory_engagement me ON m.memory_id = me.memory_id
LEFT JOIN chrome_metadata cm ON m.memory_id = cm.memory_id
LEFT JOIN gmail_metadata gm ON m.memory_id = gm.memory_id
LEFT JOIN youtube_metadata ym ON m.memory_id = ym.memory_id
WHERE m.is_deleted = FALSE
      AND m.preprocessed = TRUE
      AND ({source_filter})
      AND ({keyword_conditions})
      AND ({time_filter})
ORDER BY me.last_accessed_at DESC LIMIT 1000
"""

query = assemble_dynamic_filters(base_query, intent)
results = db.execute(query)
state.postgres_results = results
RETURN state

```

8.6 Conversation Memory Setup

```

function get_session_history(session_id: str):
    RETURN PostgresChatMessageHistory(
        connection_string = NEON_DATABASE_URL,
        session_id        = session_id
    )

function create_conversational_chain(session_id):
    history = get_session_history(session_id)
    memory = ConversationBufferMemory(
        chat_memory = history,
        return_messages = True
    )

```

```
)
```

```
RETURN memory
```

8.7 Chrome Extension — Intent Gate

So the background.js in the Chrome extension is what implements the two-phase intent gate with precise foreground time tracking.

```
function on_tab_activated(tab_id):
    active_tab = tab_id
    start_timer(tab_id)

function tick_timer():
    IF tab_is_active_tab
        AND browser_window_has_focus
        AND NOT user_idle_for_30_seconds:
            dwell_times[active_tab] += 1

    // Phase 1: Exposure gate
    IF dwell_times[active_tab] == 5:
        record_exposure_in_redis(active_tab_url)

    // Phase 2: Intent evaluation
    IF dwell_times[active_tab] >= 10:
        mark_phase2_passed(active_tab)
        send_to_backend(active_tab_url, dwell_times[active_tab])

function on_scroll(scroll_depth):
    IF scroll_depth >= 0.25 AND phase1_passed(active_tab):
        mark_phase2_passed(active_tab)
        send_to_backend(active_tab_url, dwell_times[active_tab])

function on_interaction():
    IF phase1_passed(active_tab):
```

```
mark_phase2_passed(active_tab)
```

```
send_to_backend(active_tab_url, dwell_times[active_tab])
```

Chapter 9

CONCLUSION OF CAPSTONE PROJECT PHASE 2

The phase started with getting the high-level design and system architecture done first. the 16-table PostgreSQL schema was deployed on Neon.tech with all the constraints, indexes, and foreign keys in place. OAuth 2.0 authentication was set up for both Gmail and YouTube APIs through Google Cloud Console. Redis was configured on Upstash and then the GitHub repository was set up with shared access across all four team members so that we could do parallel development.

the Gmail Connector was implemented and basically it delivers all emails through Gmail API OAuth 2.0 polling to PostgreSQL with no intent filter on receipt. the reason for that is inbox awareness needs all emails to be searchable, meaning it doesn't matter whether the user opened them or not. thread-level context enrichment was also implemented so that reply emails that have empty subjects get the keyword context from prior messages in the same thread. and then two-tier attachment handling was implemented, so the first tier is lightweight extraction of the first 500 characters at ingestion time, and the second is full on-demand extraction that is cached, making the complete retrieval pipeline functionally complete.

the result gets cached in Redis for one hour when it ranks in the top three.

the Chrome Connector was implemented using a custom Manifest V3 Chrome Extension with a two-phase intent gate. the first phase requires a minimum of five seconds of foreground exposure before any record is created. the second phase requires at least one of the following: ten or more seconds of foreground active time, scroll depth of 25% or more, at least one content interaction, or a Redis revisit signal from an earlier brief visit that day. and the foreground timer pauses on tab switch, browser minimize, or 30 seconds of user idle, so that only genuine reading time is counted.

the YouTube Connector was implemented using URL pattern detection for both regular videos and Shorts. the intent gate requires 20 or more seconds of foreground playback, any manual interaction such as pause, seek, or speed change, or a revisit signal.

metadata is fetched using the YouTube Data API v3. transcripts are fetched on a best-effort basis, and Shorts are classified by URL pattern and automatically assigned to the entertainment system group.

the Storage Engine was refactored into a centralized module, so all three connectors now route their writes through the `store_memory_item()` interface. it executes atomic transactions following Pattern A from the database design specification. no partial ingestion can be committed. the `UNIQUE(source_type, source_id)` constraint with `ON CONFLICT DO NOTHING` makes sure the system can restart safely at any time without creating duplicates.

the Enrichment Pipeline was implemented as a background worker. text cleaning removes HTML, tracking pixels, email signatures, and quoted replies from Gmail items, and it also applies the Readability algorithm to Chrome items. the four-stage cascading classifier assigns every item to one of five system groups. it first uses structural API signals, which is approximately 50% of items at zero cost, and then domain lookup, which is approximately 25% of Chrome items, nearest centroid cosine similarity against FAISS embeddings which is approximately 15%, and LLM batch fallback which is approximately 10%. Embedding generation using all-MiniLM-L6-v2 produces 384-dimensional vectors indexed in FAISS, and the `embeddable_text` is stored in the `embedding_index` table for audit and rebuild purposes.

the LangGraph Retrieval and Synthesis Engine was implemented as a nine-node stateful graph. the `parse_intent` node converts natural language queries into structured JSON using LangChain's Chat-Model abstraction configured with Gemini 2.5 Flash, and it uses few-shot examples to make sure the structured output is reliable. the `postgres_search` node executes dynamic SQL across all five joined tables. the `faiss_search` node performs filtered vector similarity search on the PostgreSQL candidate set. the `evaluate_quality` node deterministically assesses results through five sequential checks, and it does this with no LLM call at all. the `widen_scope` node progressively relaxes filters up to three times, so first it widens the time window, then it removes source filters, and then enables the full FAISS scan, before routing to `no_results_found`.

the `synthesize` node assembles context from ranked results, attachment content, and conversation history, and it generates a readable cited answer. Multi-turn conversation memory via LangChain's `PostgresChatMessageHistory` allows Turn 2 queries to resolve references from Turn 1.

So at the end of Phase 2, the system accepts a natural language query and parses it into structured intent. It then executes cross-source retrieval across Gmail, Chrome, and YouTube data. It evaluates the result set and widens it up to three times if needed, and then synthesizes a contextual answer citing specific sources with timestamps. That is the complete retrieval pipeline, and it will be ready for integration with the remaining modules in Phase 3.

Chapter 10

PLAN OF WORK FOR CAPSTONE PROJECT PHASE 3

Phase 3 will complete the remaining implementation deliverables and stabilize the full Echo system.

10.1 Digital Wellbeing and Analytics Module (WBA)

The wellbeing module will implement time aggregation queries that compute time spent per system category from engagement data without storing aggregated values separately. Session computation will identify contiguous blocks of activity, meaning consecutive items within a five-minute gap, and store them in the sessions and session_memory_map tables. The user groups feature will implement the hybrid KNN plus rule classification engine with human-in-the-loop approval. It will go through the group_suggestions table, the group_rules persistent rule engine, and the weekly batch review interface. The regret system will implement per-item and per-session regret marking, and then pattern detection by time of day and category, and also rate-limited regret reminders. LLM-generated weekly insights will be produced from aggregated numeric data only and not from raw email bodies, page content, or video transcripts.

10.2 React Dashboard and User Interface Layer (UIL)

The web dashboard will implement all views. So first the unified search interface with synthesized answer display and source citation cards, and then the daily summary with system category time breakdown, the time heatmap grid with regret markers, and the source breakdown analytics, the user groups management interface with the weekly review queue, the regret analytics charts, and also the settings panel for LLM API key configuration and source enable/disable. The Chrome extension popup will expose a quick search bar that feeds the full LangGraph RSE pipeline, a list of recent items across all sources, and a pause/resume tracking toggle.

10.3 End-to-End Integration and Testing

So phase 3 will go through all the test scenarios specified in the architecture document. That is Gmail ingestion tests, Chrome intent gate tests, YouTube intent gate tests, LangGraph single-turn and multi-

turn query tests, self-correcting loop tests, and the performance benchmarks. And then a privacy audit will verify that no personal data leaves the local machine through any unintended code path.

10.4 Migration and Submission Preparation

The `pg_dump` migration from Neon.tech to local PostgreSQL will be executed and validated.

The Docker shared environment image will be finalised. And then the final report will be completed and submitted through the PESU project repository server along with the plagiarism check report.

10.5 Timeline

The target for completion of all remaining modules is November 2026. This includes the Digital Wellbeing module, the complete React dashboard, the Chrome Extension popup, end-to-end integration, the full test suite execution, and performance validation. And then the period after November 2026 will be used for bug fixes, optimisation, privacy audit, and the final report preparation ahead of submission.

Bibliography

- [1] Y. Zhang, S. Liu, and J. Wang, “Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL,” in *Proc. IEEE ICDE*, 2024, pp. 3640–3653.
- [2] R. Islambouli, S. Ingram, and D. Gillet, “Understanding Digital Wellbeing Through Smartphone Usage Intentions and Regrettable Patterns,” in *Proc. IEEE ICHI*, 2024.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] M. Kleppmann et al., “Local-First Software: You Own Your Data, in Spite of the Cloud,” in *Proc. Onward!*, 2019, pp. 154–178.
- [6] V. Kostakos, A. Giannoulis, and M. Langheinrich, “Privacy-Preserving Data Storage and Processing: A Survey of Modern Approaches,” *ACM Comput. Surv.*, vol. 54, no. 6, 2021.
- [7] Y. Sun, A. Dubey, and E. P. Xing, “Lifelong Learning with Retrieval-Augmented Memory,” in *Proc. NeurIPS*, vol. 33, 2020.

Glossary

Echo module — this is the one responsible for capturing Chrome browsing activity through the Manifest V3 extension.

ENP — Enrichment Pipeline. So this is the background worker that cleans, classifies, and embeds raw captured items.

FAISS — Facebook AI Similarity Search. Basically the library used for fast approximate nearest-neighbour search over embedding vectors.

GMC — Gmail Connector. The Echo module that is responsible for capturing Gmail data via the Gmail API.

KNN — K-Nearest Neighbours. Used in the user group classifier to get the K most similar existing group members to a new item.

LLM — Large Language Model. In Echo, used for intent parsing (LLM Call 1) and answer synthesis (LLM Call 2) per query, and it goes through LangChain's ChatModel abstraction to do that.

MV3 — Manifest Version 3. So this is the current Chrome Extension manifest format that the Echo Chrome Extension uses.

OAuth — Open Authorisation. The authentication protocol that Echo uses to access Gmail and YouTube APIs without storing user credentials.

RSE — Retrieval and Synthesis Engine. So this is the LangGraph-based module that handles natural language queries and produces answers.

STE — Storage Engine. The centralised module that handles all PostgreSQL, FAISS, and Redis write operations.

TTL — Time-To-Live. The expiry duration of keys in Redis. Echo uses 24 hours for revisit signals and 1 hour for attachment text cache.

UIL — User Interface Layer. So this is the React dashboard, Chrome Extension popup, and FastAPI gateway.

WBA — Digital Wellbeing and Analytics module.

YTC — YouTube Connector. The Echo module responsible for capturing YouTube watch activity.

EchoState — The TypedDict state object passed between nodes in the LangGraph RSE graph.

Dwell Time — The total cumulative foreground time spent on a Gmail or Chrome item, and that is basically the time where the tab is active, the browser is focused, and the user is not idle for 30 seconds.

Intent Gate — The filtering logic that determines whether a browsing or viewing activity should be saved, using engagement signals specific to each source type.

Memory Item — A single captured unit of digital activity. So that is one email, one web page, or one YouTube video, and it is represented as a row in `memory_items` and its associated subtype tables.

Preprocessed — The boolean flag on `memory_items` that indicates whether the Enrichment Pipeline has completed all steps for that item. The steps are cleaning, classification, and embedding, and FAISS indexing.

System Group — One of five fixed coarse category that the four-stage classifier assigns to every memory item. So the categories are work, study, and entertainment, personal, and misc, and it should never be NULL.

User Group — A user defined custom categories for example Capstone Project or Placement Preparation. It is managed through the hybrid KNN plus rule engine with mandatory human in the loop approval.